CS 5330 Final Project: Transfer Learning versus Random Initialization

Isolating feature quality from trainable capacity in scene classification.

Team

- **Shriman Raghav Srinivasan**, srinivasan.shrim@northeastern.edu
- **Group members: none.** This is an individual project. All code, experiments, report, and slides are my own work.
- Khoury College of Computer Sciences, Northeastern University
- CS 5330 Pattern Recognition and Computer Vision, Summer 2026, Prof. Bruce Maxwell

Demo Video and Presentation

Demo video (2 min 5 sec, 1080p, silent with on-screen captions):

`https://drive.google.com/file/d/1JZSamG6WljxIDFxSjuwxOERkdl2lQ34F/view?usp=sharing`

Final presentation (12 slides):

`https://drive.google.com/file/d/1ImzhkgC75xF5Fq-7q1qQB8d4SKsGyZQx/view?usp=sharing`

Both files are also in this repo, at `presentation/demo_video.mp4` and `presentation/Final_Presentation-PRCV.pdf`, and both URLs appear in the compiled report under Supplementary Material.

Dataset: Intel Image Classification on Kaggle. Direct zip mirror: HuggingFace.

Project Description

Everyone knows transfer learning beats training from scratch. What nobody checks is *why*, because the usual experiment moves two things at once: the quality of the pretrained features, and how many parameters you left trainable. Freeze the early layers of a pretrained network and it wins. Was that the ImageNet knowledge in the frozen weights, or just an easier optimization problem with fewer parameters to fit on a small dataset? Both stories predict the same headline number, so the comparison everyone runs cannot tell them apart.

This project adds the control that separates them. Same ResNet-18, same six-class scene dataset, three conditions that differ only in how the early block θ_f is treated:

Condition	Early block θ_f	Trainable	Why it is here
Pretrained-Frozen (PT-F)	ImageNet weights, frozen	`layer4` + `fc`, 8,396,806	the transfer-learning arm
Random-Frozen (R-F)	random weights, frozen	`layer4` + `fc`, 8,396,806	**the capacity-matched control**
Random-Full (R-FL)	random weights, trained	everything, 11,179,590	the from-scratch arm

Random-Frozen is the whole point. It trains the exact same 8,396,806 parameters as Pretrained-Frozen and differs in one thing only: whether the frozen weights came from ImageNet or from `torch.randn`. So PT-F

minus R-F is feature quality with capacity nailed down, and R-F minus R-FL is capacity with initialization nailed down. Two clean measurements instead of one confounded one.

What I set out to answer

1. How much is feature quality worth once capacity is fixed? PT-F against the capacity-matched control.
2. How much is capacity worth on its own? The frozen random control against a fully trainable network.
3. Where does fine-tuning stop paying? A freeze-depth sweep from linear probe up to full fine-tuning.
4. How far up the network does the first layer still matter? Swap `conv1` for a fixed Gabor bank, then unfreeze upward one stage at a time.

The bug that would have silently ruined this. `requires_grad = False` does not freeze BatchNorm. The running statistics keep updating on every forward pass, so a block you believe is frozen quietly drifts and the whole capacity-matching argument falls apart. Both frozen conditions hold BatchNorm in `eval()` mode. Random-Full updates normally, which is correct for that arm.

Protocol. Stratified 90/10 train and validation split (12,631 and 1,403 images), 3,000 held-out test images. SGD at lr 1e-3, momentum 0.9, weight decay 1e-4, batch size 32. Three conditions by three seeds (42, 100, 2026), early stopping on validation loss with patience 5. Nine runs, each evaluated once on the test split.

Results

Condition	Mean test acc.	Std. dev.	Epochs	Wall clock
Pretrained-Frozen	**93.27%**	0.23%	8.0 ± 0.8	103 s
Random-Full	84.69%	1.87%	15.0 ± 2.2	426 s
Random-Frozen	66.54%	4.04%	8.7 ± 2.1	113 s

Pretrained features are worth 26.7 points at matched capacity. Identical trainable parameters, identical data, identical recipe. The only difference is where the frozen weights came from.

Capacity on its own is worth 18.1 points, and buying it costs you: Random-Full still lands 8.6 points below Pretrained-Frozen while taking roughly four times the wall clock. Both factors are real. The representation is the bigger one by a wide margin.

Freeze-depth ablation. A bare linear probe on frozen pretrained features already hits **91.3% from 3,078 trainable parameters**. Unfreezing `layer4` adds 1.6 points to 92.9%. Unfreezing `layer3`, `layer2`, or the entire network gives 92.8% every time, so the elbow sits exactly at `layer4` and everything past it is wasted compute. Most of what pretraining gives you needs almost no fine-tuning to collect. (40% training subset, seed 42.)

Gabor first layer. Replacing `conv1` with a fixed bank of 8 orientations by 4 wavelengths by 2 phases costs about 10 points at read-out, 81.1% against 91.6%. Letting `layer1` adapt returns 5 of those points. Letting `layer2` through `layer4` adapt returns nothing. **Roughly 7 points never come back**, so a hand-designed filter bank plus unlimited downstream retraining cannot reconstruct what a learned first layer provides. (Full data, seed 42.)

Where the errors are. Glacier against Mountain, and Buildings against Street. Same two pairs in every condition, structurally identical and just larger under random initialization. That is genuine visual ambiguity in the data, not something broken in the training setup.

Repository Contents

Graded deliverables

What	Where
Report, 7 pages, IEEE format	`report/report.pdf`
This README as a PDF	`report/README.pdf`
Presented slide deck	`presentation/Final_Presentation-PRCV.pdf`
Demo video	`presentation/demo_video.mp4`

Experiment outputs

What	Where
Trained checkpoints, 9 runs	`results/best_model_{condition}_seed{seed}.pth`
Main metrics	`results/summary_metrics.json`, `results/test_evaluation_metrics.json`
Ablation metrics	`results/ablation/ablation_results.json`, `gabor_ablation_results.json`
Per-run training curves	`results/history_{condition}_seed{seed}.json`

Seven figures, all under `results/`:

```
loss_curves.png       accuracy_curves.png    confusion_matrices.png
learning_curves.png   feature_pca.png        ablation_curve.png
gabor_curve.png
```

About the dataset. Intel Image Classification: 150x150 natural scene images across six classes (buildings, forest, glacier, mountain, sea, street), 14,034 training and 3,000 test images. An unlabeled `seg_pred/` split ships with it but I do not use it here.

Layout

```
final_project/
|-- data/                     # seg_train/, seg_test/, seg_pred/
|-- src/
|   |-- dataset.py            # transforms, stratified validation split
|   |-- model.py              # ResNet-18 setup, freezing, BN eval enforcement
|   |-- train.py              # training loop, early stopping
|   |-- evaluate.py           # test split, precision/recall/F1 in plain numpy
|   |-- run_experiments.py    # the 9-run grid
|   |-- visualize.py          # curves and confusion-matrix heatmaps
|   |-- generate_latex_results.py # metrics JSON into LaTeX macros
|   |-- ablation_freeze_depth.py  # freeze-boundary sweep
|   |-- gabor_ablation.py         # fixed Gabor conv1 vs learned conv1
|   `-- make_extra_figures.py     # learning curves, PCA, both ablation curves
|-- results/                  # checkpoints, metrics, figures
|   `-- ablation/             # JSON and logs for both extensions
|-- report/                   # LaTeX source, bibliography, compiled PDFs
|-- presentation/             # deck, assets, demo video, generators
|-- proposal/                 # original project proposal
|-- main.py                   # --run / --eval / --compile
`-- verify_pipeline.py        # 5-second pre-flight check
```

Every number in the report comes out of the metrics JSON through `generate_latex_results.py`, which writes LaTeX macros. The report cannot silently disagree with the runs.

Running It

1. Requirements

```
pip install torch torchvision numpy matplotlib
```

Rebuilding the generated deck also needs `python-pptx` and `Pillow`, plus LibreOffice for the PPTX to PDF step. Compiling the report needs `pdflatex` and `bibtex`. Building the demo video needs `ffmpeg` with libx264.

```
pip install python-pptx Pillow
```

2. Dataset

Download from either link above and put the extracted `seg_train/`, `seg_test/`, and `seg_pred/` folders under `data/`. There is no automatic download step.

3. Pre-flight check

Takes about 5 seconds. Verifies model partitioning, batch norm handling, and that the environment works before you burn GPU hours:

```
python3 verify_pipeline.py
```

4. The full grid

Nine runs, 3 conditions by 3 seeds, with early stopping. Checkpoints and curves land in `results/`:

```
python3 main.py --run
```

5. Evaluation and plots

```
python3 main.py --eval
```

Writes `results/summary_metrics.json` and `results/test_evaluation_metrics.json`, and plots `loss_curves.png`, `accuracy_curves.png`, and `confusion_matrices.png`.

6. The two ablations

These sit outside the `main.py` grid because their compute profiles are different. Run them as modules from the project root. Output goes to `results/ablation/`.

Freeze-depth sweep, `fc` linear probe through `layer4`, `layer3`, `layer2`, to full fine-tune. Defaults to a 40% stratified training subset at one seed to stay tractable on CPU. On a GPU use `-frac 1.0 -seeds 42 100 2026`:

```
python3 -m src.ablation_freeze_depth
```

Fixed Gabor first layer against the learned one, swept over unfreeze depth, full data:

```
python3 -m src.gabor_ablation --seeds 42
```

7. The supplementary figures

Learning curves and the feature PCA come from existing checkpoints with no retraining. The two ablation curves come from the JSON written in step 6:

```
python3 -m src.make_extra_figures              # learning_curves.png, feature_pca.png
python3 -m src.make_extra_figures --ablation   # ablation_curve.png
python3 -m src.make_extra_figures --gabor      # gabor_curve.png
```

8. Report and slides

```
python3 main.py --compile
```

Writes `report/results_summary.tex` from the metrics JSON, runs `pdflatex` and `bibtex` for `report/report.pdf`, then builds `presentation/presentation.pptx` via `make_pptx.py` and converts it with headless LibreOffice. The conversion is skipped with a warning if `soffice` is not on your PATH.

Slide figures regenerate separately when the underlying results change:

```
python3 presentation/generate_assets.py
```

To rebuild `report/README.pdf` after editing this file:

```
python3 report/make_readme_pdf.py
```

9. The demo video

Renders `presentation/demo_video.mp4` at 1920x1080, 30 fps, about two minutes, silent with burned-in captions. Needs `ffmpeg` with libx264 plus the checkpoints and figures from steps 5 and 7:

```
python3 presentation/make_demo_video.py
```

The first segment loads the Pretrained-Frozen and Random-Frozen checkpoints at seed 42 and runs both over the same held-out `seg_test` images, showing the full softmax for each. Since both train an identical 8,396,806 parameters, every disagreement you see is down to initialization alone. The images are a fixed-seed random draw, not hand-picked. The second segment walks the result figures. Flags: `-n-images`, `-fps`, `-seed`, `-dry-run`.

Limitations

Section VI of the report covers these in full. In short:

- **One dataset, one architecture.** I am not claiming the 26.7-point figure transfers to other domains or deeper networks.
- **Three seeds, no significance test.** The gaps between conditions are far larger than the spread within them, but I did not run a formal test. Both ablations are single-seed, so read the shape of those curves rather than their exact values.
- **No data augmentation.** One deterministic resize and centre crop, shared by all three splits. That keeps the conditions on an identical input distribution, but it holds absolute accuracy below tuned baselines and it penalises Random-Full most, since a fully trainable network is the arm with the most to gain from augmentation.
- **The Gabor bank is untuned.** I fixed its parameters up front instead of searching them, so the leftover 7-point gap is an upper bound.
- **Random-Frozen is a control, not a method.** It exists to hold capacity fixed. Nobody should deploy a 66.5% model.

Acknowledgments

Prof. Bruce Maxwell (CS 5330, Khoury College). His question about how far the first layer's influence actually reaches up the network is what became the Gabor experiment. The Intel Image Classification dataset is distributed publicly via Kaggle. The ResNet-18 implementation and the ImageNet-pretrained weights come from PyTorch and torchvision.